

CHAPTER 17

17. 완성형 쇼핑몰 — 전체가 맞물리는 자리

Node.js · Express · NestJS 강의 · 2026

이 장은 손으로 짜지 않는 자료제공 장이다. 10~16장에서 만든 것을 한 앱으로 모은 완성본에 검색과 미결제 자동취소를 더 하고, 13장부터 간단한 조건문으로 하던 역할 검사를 `@Roles + RolesGuard` 로 정리하는 심화 기법을 본다. 돌려보고 코드를 읽으며 전체가 어떻게 맞물리는지 확인한다.

학습 목표

이 장은 손으로 짜지 않고 읽고 돌려보는 자료다. 이 장을 마치면 다음을 할 수 있다.

- 앞 장에서 배운 기술이 한 앱에서 어떻게 맞물리는지 설명할 수 있다.
- LIKE 기반 검색의 동작과 한계를 설명할 수 있다.
- `@Cron` 으로 미결제 주문을 자동 취소하고 재고를 복구하는 흐름을 읽을 수 있다.
- 반복되는 역할 검사를 `@Roles + RolesGuard` 로 정리하는 방법을 이해할 수 있다.

돌려 보기

```
docker compose up -d

cd 17
npm install
cp .env.example .env
npx prisma migrate dev --name init
npm run seed
npm run start:dev          # Swagger 문서: http://localhost:3000/docs
```

/docs 를 열면 지금까지 만든 모든 엔드포인트가 한눈에 보인다. 회원가입부터 배송까지, 보호 라우트에는 자물쇠 표시가 붙어 있다.

검색 — LIKE로 상품명 찾기

상품 목록에 검색어를 더했다. Prisma의 `contains` 는 PostgreSQL의 `ILIKE` 로 번역된다 — 대소문자를 가리지 않는 부분 일치다.

```
const where = {
  ...(search ? { name: { contains: search, mode: "insensitive" } } : {}),
  ...(categoryId ? { categories: { some: { id: categoryId } } } : {}),
};
```

search=이어폰 이면 `WHERE name ILIKE '%이어폰%'` 가 된다. 검색어와 분류 필터를 함께 적용할 수도 있다.

실무 메모. `ILIKE '%...%'` 는 입문 단계에 가장 직관적이지만, 데이터가 커지면 인덱스를 잘 못 타 느려진다. 그때는 PostgreSQL의 전문 검색(`tsvector` , `pg_trgm`)이나 검색 엔진(OpenSearch 등)으로 옮긴다. 지금 단계에서는 `LIKE`로 충분하다.

미결제 주문 자동취소 — @Cron의 실전

장바구니에 담아 주문까지 했는데 결제하지 않고 묶여 있는 주문이 있다. 이런 주문이 재고를 잡고 있으면 다른 사람이 살 수 없다. 그래서 일정 시간이 지난 미결제 주문을 자동으로 취소하고, 잡아 둔 재고를 풀어 준다.

```
@Cron(CronExpression.EVERY_MINUTE)
async cancelStalePendingOrders() {
  const timeoutMin = Number(process.env.PENDING_ORDER_TIMEOUT_MIN ?? 30);
  const cancelled = await this.ordersService.cancelStalePending(timeoutMin);
  if (cancelled > 0) {
    this.logger.log(`미결제 ${timeoutMin}분 경과 주문 ${cancelled}건 자동 취소, 재고 복구 완료`);
  }
}
```

14장의 통계 스냅샷과 같은 `@Cron` 이지만, 이번엔 주문이라는 실제 데이터를 다룬다. 취소는 15장에서 만든 "재고 복구 + 상태 변경" 트랜잭션을 그대로 재사용한다. 수동 취소와 자동 취소가 같은 로직을 공유하는 것이다.

심화 — 역할 검사를 @Roles + RolesGuard로

13장부터 역할 검사를 간단한 `if` 로 했다.

```
if (user.role !== "SELLER" && user.role !== "ADMIN") {
  throw new ForbiddenException("판매자만 상품을 등록할 수 있습니다.");
}
```

눈에 보이고 이해하기 쉽지만, 라우트가 늘면 같은 `if` 가 여기저기 반복된다. NestJS에는 이 반복을 한 줄로 줄이는 방법이 있다. 라우트에 "이 역할만 들어올 수 있다"는 표식을 달고(`@Roles`), 그 표식을 읽어 막는 가드(`RolesGuard`)를 두는 것이다.

```
// 표식을 다는 데코레이터
export const Roles = (...roles: Role[]) => SetMetadata(ROLES_KEY, roles);

// 표식을 읽어 판단하는 가드
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly reflector: Reflector) {}
  canActivate(context: ExecutionContext): boolean {
    const required = this.reflector.getAllAndOverride<Role[]>(ROLES_KEY, [
      context.getHandler(), context.getClass(),
    ]);
    if (!required || required.length === 0) return true;
    const { user } = context.switchToHttp().getRequest();
    if (!user || !required.includes(user.role)) {
      throw new ForbiddenException("이 작업을 수행할 권한이 없습니다.");
    }
    return true;
  }
}
```

이제 컨트롤러는 `if` 없이 데코레이터만 단다.

```
@Post()
@UseGuards(JwtAuthGuard, RolesGuard)
@Roles(Role.SELLER, Role.ADMIN)
create(@Body() dto, @CurrentUser("id") sellerId) {
  return this.productsService.create(dto, sellerId);
}
```

`Reflector` 는 라우트에 붙은 메타데이터(`@Roles` 가 단 표식)를 읽어 오는 도구다. 가드가 그 표식과 토큰의 역할을 비교해 통과/거부를 정한다. 가드는 순서대로 도니까, `JwtAuthGuard` 가 먼저 `req.user` 를 채우고 그다음 `RolesGuard` 가 역할을 본다.

`if` 와 `@Roles` 중 무엇이 옳다기보다, 무엇을 먼저 배우느냐의 문제다. `if` 는 동작이 눈에 보이고, `@Roles` 는 반복을 줄인다. 개념을 익힌 뒤에 도구로 정리하는 순서가 자연스럽다.

전체 흐름 한눈에

가입·로그인 → 토큰(역할 포함)

판매자: 상품 등록(+이미지) → 상품 상세는 캐시

구매자: 장바구니 담기 → 주문(트랜잭션: 재고 차감) → 결제(PAID)

관리자: 분류 관리, 배송 상태머신(PAID→SHIPPED→DONE)

스케줄러: 미결제 주문 자동취소(재고 복구), 일별 통계 스냅샷(상품 수·재고·결제 건수·매출)

각 칸은 앞 장에서 하나씩 배운 것이다. 인증은 13장, 캐시는 14장, 트랜잭션은 15장, 이미지·상태머신은 16장. 17장은 이들을 잇는 배선과 마지막 손질이다.

실무로 가기 전에 더 챙길 것

이 쇼핑물은 학습용으로 완결됐지만, 실제 서비스로 올리려면 더 챙길 게 있다. 다음 단계의 지도로 남겨 둔다.

- **리프레시 토큰** — 지금은 한 시간짜리 액세스 토큰뿐이다. 만료되면 다시 로그인해야 한다. 실무는 짧은 액세스 토큰 + 긴 리프레시 토큰으로 세션을 잇는다.
- **환경 변수 검증** — `@nestjs/config` 로 환경 변수를 한곳에서 관리하고, 시작할 때 빠진 값이 없는지 검증한다.
- **전역 예외 필터** — 에러 응답 형식을 한 가지로 통일한다.
- **테스트** — 서비스 단위 테스트와 e2e 테스트로 흐름을 자동 검증한다.
- **결제 연동** — 모의 결제를 실제 PG사 연동 + 웹훅으로 바꾼다.

정리

10장의 상품 한 줄이 17장의 쇼핑물이 됐다. 같은 도메인을 한 장씩 쌓아 올린 덕에, 새 기술이 들어올 때마다 "이게 어디에 붙는지"가 분명했다. 모듈·DI에서 시작해 Prisma, 인증, 캐시, 트랜잭션, 파일, 상태머신까지 — 각각은 작은 한 걸음이었지만, 모으니 실전 앱이 됐다. 역할 검사처럼 처음엔 `if` 로 익히고 나중에 `@Roles` 로 정리하듯, 기술도 개념을 먼저 잡고 도구로 다듬으면 된다. 이 흐름이 NestJS로 실제 서비스를 만들 때 그대로 이어진다.